

Great choice. The original Game Boy (DMG-01) is one of the most rewarding systems to emulate. It's small enough that one person can understand every single wire in the machine, but complex enough to feel like a "real" computer.

Here is your step-by-step roadmap for building a Game Boy emulator.

1. The Core Architecture

The Game Boy is built around the **LR35902** CPU. It's a mix between an Intel 8080 and a Zilog Z80.

- **Registers:** You'll need to model 8-bit registers: A, B, C, D, E, H, and L.
- **Combined Registers:** Some instructions treat them as 16-bit pairs: AF, BC, DE, and HL.
- **The Flags:** The F register stores CPU state: **Zero**, **Subtract**, **Half-Carry**, and **Carry**.
- **Memory Map:** It uses a 16-bit address space (\$0000\$ to \$FFFF\$).

2. Development Roadmap

Phase 1: The Memory Bus

Don't start with the CPU; start with the **Bus**. Create a class or module that handles `read(address)` and `write(address, value)`.

- Map \$0000\$–\$7FFF\$ to the ROM.
- Map \$8000\$–\$9FFF\$ to Video RAM (VRAM).
- Map \$FF80\$–\$FFFE\$ to High RAM (HRAM).
- **Goal:** Be able to load a .gb file into an array and read the first few bytes.

Phase 2: The CPU (The Big One)

This is where you spend 70% of your time. You need to implement the **Fetch-Decode-Execute** cycle.

1. **Fetch:** Read the byte at the Program Counter (\$PC\$).
2. **Decode:** Look up what that byte means (e.g., 0x06 is LD B, n).
3. **Execute:** Modify the registers and increment the \$PC\$.
4. **Prefixes:** If you see 0xCB, you must fetch the *next* byte and look it up in a different table of instructions (the "Bitwise" instructions).

Phase 3: The PPU (Graphics)

The Game Boy doesn't have a "frame buffer" where you just draw pixels. It draws line-by-line using **Tiles**.

- **Background:** 8x8 pixel tiles stored in VRAM.
- **Scanlines:** The screen is 144 pixels tall. You need to track which scanline is currently being "drawn" and trigger a **V-Blank** interrupt when you reach the bottom.
- **Palettes:** Map the values 0, 1, 2, 3 to four shades of green/gray.

Phase 4: Timers and Interrupts

The Game Boy is alive. It has a clock that ticks constantly.

- **The Divider (DIV):** A register that increments at a fixed rate. Many games use this for random numbers.
- **V-Blank Interrupt:** This tells the game "I'm done drawing the screen, you can move objects now." Without this, games will just freeze.

3. Essential "Secret" Resources

You cannot do this with Wikipedia alone. Use these specific documents:

- [The Pan Docs](#): The absolute bible of Game Boy development. If it's not in here, it doesn't exist.
- [Game Boy CPU Manual](#): A cleaner breakdown of every single opcode and how it affects the flags.
- [Blargg's Test ROMs](#): These are ROMs designed to fail if your CPU logic is even 1 bit off. They are your best friend.

4. Common Pitfalls to Avoid

- **The Half-Carry Flag:** This is the most annoying part of the CPU. It checks if a carry occurred from bit 3 to bit 4. It's only used for BCD (Binary Coded Decimal) math. Look up a helper function for this; don't try to "logic" it out every time.
- **The BIOS:** You don't *need* the original Nintendo Boot ROM (the scrolling logo) to start. You can "fake" the state of the registers as they would be after the BIOS finishes and jump straight to the game code at address \$0100.
- **Signed vs Unsigned:** Some instructions use signed 8-bit integers (e.g., JR jumps). Make sure your language handles the conversion from 0xFF to -1 correctly.

Your First Milestone

Build a "Disassembler." Load a ROM and print the names of the instructions to the console. Once you see LD SP, \$ffff and JP \$0150 printing out, you know your memory and fetch logic are working!

Which programming language have you settled on? (I can give you specific tips for C++, Rust, or Python/JavaScript based on your choice).